Hangman code

Binnen dit project heb ik hier bijgehouden wat hier alles doet, normaal gesproken hoor je niet voor alles comments te gebruiken en het is ook niet de bedoeling dat het voor elke functie te zien is, maar ik heb een tutorial filmpje gevolg van iemand anders en ik werk niet vaak met python daarom was het belangrijk voor mij om comments te gebruiken. Vooral omdat ik dit project niet regelmatig zal gebruiken, aangezien ik nog andere projecten tussendoor maak.

hangman_app.py

hangman_app.py > ...

```python
import streamlit as st
import random

# Hulpfuncties
def kies_woord(woordenlijst):
    """Kiest een willekeurig woord uit de lijst."""
    return random.choice(woordenlijst).lower()

def maak_nieuw_spel(woordenlijst):
    """Initialiseer session_state voor een nieuw spel."""
    woord = kies_woord(woordenlijst)
    st.session_state['woord'] = woord
    # Initialiseer met underscores voor de lengte van het woord
    st.session_state['geraden_letters'] = ["_"] * len(woord)
    st.session_state['fouten'] = 0
    # Het aantal stappen in de tekening is 6 (0 t/m 6 is 7 stappen)
    st.session_state['max_fouten'] = 7
    st.session_state['geraden_set'] = set()  # onthoud welke letters al geraden zijn
    # Reset het inputveld
    st.session_state['gok_input'] = ""

```

import streamlit as st: Hiermee wordt de bibliotheek geladen die ervoor zorgt dat de code als een interactieve website kan draaien.

import random: Dit is nodig om straks een willekeurig woord uit een lijst te kunnen kiezen.

Met **def** stel je een variabele vast en kan je kiezen wat het voor functie word als het later opnieuw gaan aanroepen

```python
# 1. Haal de letter op uit de state
letter = letter.lower().strip()

# Valideer de invoer
if not letter.isalpha() or len(letter) != 1:
    st.warning("Ongeldige invoer. Voer alstublieft één geldige letter in (a-z).")
    # Reset de input zodat de gebruiker opnieuw kan typen
    st.session_state['gok_input'] = ""
    return

# 2. Check of al geraden
if letter in st.session_state['geraden_set']:
    st.info(f"Je hebt '{letter.upper()}' al geprobeerd.")
else:
    # 3. Voeg toe aan de set van geraden letters
    st.session_state['geraden_set'].add(letter)

    # 4. Controleer of het een juiste gok is
    if letter in st.session_state['woord']:
        # plaats letter op juiste posities
        found = False
        for i, ch in enumerate(st.session_state['woord']):
            if ch == letter:
                st.session_state['geraden_letters'][i] = letter
                found = True
        if found:
            st.session_state['last_feedback'] = f"✅ Goed geraden! De letter '{letter.upper()}' zit in het woord."
    else:
        # Foute gok
        st.session_state['fouten'] += 1
        remaining = st.session_state['max_fouten'] - st.session_state['fouten']
        st.session_state['last_feedback'] = f"❌ Fout! De letter '{letter.upper()}' zit er niet in. Nog {remaining} kans(en)."
```

Dit deel van de code is het **"belangrijkste" van het spel**: de logica die bepaalt wat er gebeurt zodra een speler een letter invult en op enter drukt.

Validatie: Er wordt gecontroleerd of de invoer wel een letter is (isalpha) en of het er precies één is. Is dat niet zo? Dan krijgt de gebruiker een waarschuwing (st.warning) en stopt de functie direct (return).

De code kijkt in de geraden_set. Als je de letter al eens hebt getypt, krijg je een melding dat je die letter al geprobeerd hebt.

Als de letter **niet** in het woord zit:

Fouten += 1: De foutenteller gaat met 1 omhoog.

Kansen berekenen: Er wordt uitgerekend hoeveel beurten je nog over hebt (max_fouten - fouten).

hangman_app.py

hangman_app.py > teken_galgje

```python
def teken_galgje(fouten, max_fouten):
        +---+
        |   |
        |
        |
        |
        |
        |
      -----
    """,
    # Fouten = 2: Hoofd
    """
        +---+
        |   |
        |   O
        |
        |
        |
        |
      -----
    """,
    # Fouten = 3: Lichaam
    """
        +---+
        |   |
        |   O
        |   |
        |   |
        |
        |
      -----
    """,
    # Fouten = 4: Linkerarm
    """
        +---+
        |   |
        |   O
        |  /|
        |   |
        |
        |
      -----
    """,
    # Fouten = 5: Rechterarm
    """
        +---+
        |   |
        |   O
        |  /|\\
        |   |
        |
        |
      -----
    """,
    # Fouten = 6: Linkerbeen
    """
        +---+
        |   |
        |   O
        |  /|\
        |   |
        |  / /
        |
      -----
    """,
    # Fouten = 7 (MAX): Rechterbeen (Verloren)
    """
        +---+
        |   |
        |   O
        |  /|\\
        |   |
        |  / \\
        |
      -----
```

Hier word de layout gevormd en dus het lichaam voor elke keer als je het goed of fout hebt komt er iets bij of blijft het zo staan

```python
    # Gebruik de juiste index (stages heeft 7+1 = 8 elementen, max_fouten = 6)
    # We gebruiken 7 stages voor 6 max fouten, dus fouten + 1 is de index.
    stage_index = min(fouten, max_fouten + 1)

    # Geef de ASCII-art terug als code-blok voor monospace font
    return stages[stage_index]

# --- Woordenlijst (bestaat uit de woorden van de gebruiker) ---
WOORDEN = [
 "aardbei", "vogelhuis", "koffie", "regenboog", "bibliotheek",
    "fietsenrek", "avontuur", "chocolade", "wiskunde", "telefoon",
    "zeilboot", "paraplu", "goudvis", "piano", "wandeling",
    "kameleon", "olifant", "appeltaart", "klokkijken", "sneeuwvlok",
    "kruispunt", "ziekenhuis", "rekenmachine", "fototoestel", "speeltuin",
    "lantaarn", "broodrooster", "tandenborstel", "bioscoop", "schaapskooi",
    "spinazie", "boekenkast", "schatkist", "watermeloen", "vuurtoren",
    "grappig", "ontbijt", "verjaardag", "tekening", "microfoon",
    "zonnepaneel", "trampoline", "pinguin", "helikopter", "diamant",
    "spinneweb", "gitaarles", "raketlancering", "vliegtuig", "kattenbak",
    "limonade", "dinosaurus", "hangmat", "keukenkast", "astronaut",
    "sushi", "ijskoffie", "woestijn", "telescoop", "spijkerbroek",
    "windmolen", "zaklamp", "museum", "puzzelstuk", "flamingo",
    "regenjas", "krantenwijk", "robotica", "schoolbord", "komkommer",
    "accordeon", "satelliet", "tandarts", "chocolademelk", "zwembad",
    "ankerplaats", "zeezout", "buitenwijk", "gelukzalig", "spiegelbeeld",
    "muurbloem", "samenhang", "haringzaak", "onweersbui", "zeeschuim",
    "zandkasteel", "pijlsnel", "groentesoep", "eindexamen", "luchtballon",
    "vleermuis", "aardappel", "brievenbus", "camping", "dolfijn",
    "energie", "fantasie", "geheimzinnig", "hologram", "inktvis",
    "jungle", "krokodil", "lava", "mandarijn", "noodlanding", "oranje", "paddestoel",
    "quarantaine", "robotarm", "sterrenhemel",

]
```

Hier is ook de lijst aan woorden die de computer op willekeurige volgorde kiest om een nieuw spel te starten.